

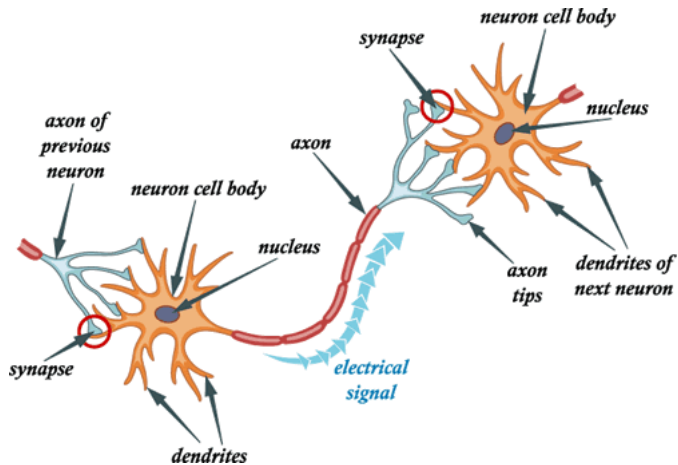
Deep Learning

Vazgen Mikayelyan

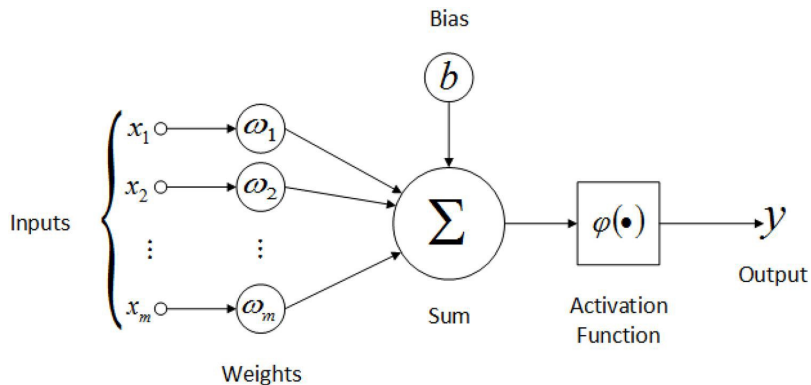


- 1 What is a neural network?
- 2 Gradient Descent
- 3 Linear and Logistic Regressions

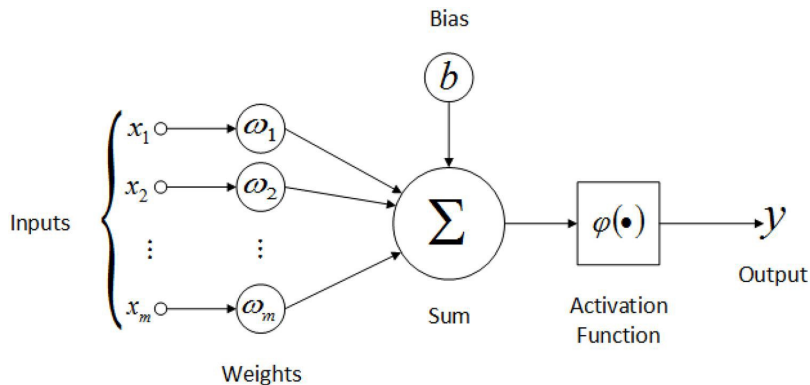
Human Brain



Artificial neuron



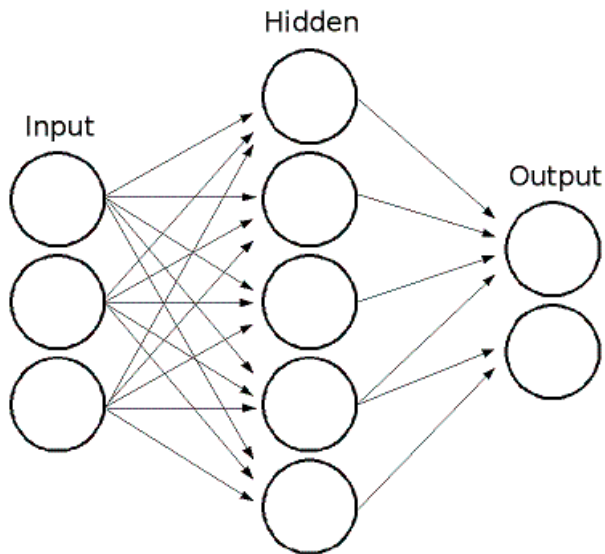
Artificial neuron



So artificial neuron is a function from $\mathbb{R}^m$ to $\mathbb{R}$, where m is the dimension of input:

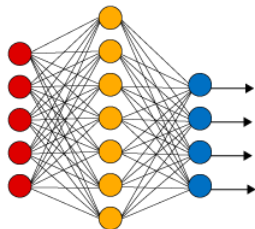
$$f_{w,b}(x_1, x_2, \dots, x_m) = \varphi(w_1x_1 + w_2x_2 + \dots + w_mx_m + b) = \varphi(w^T x + b)$$

Hidden Layer



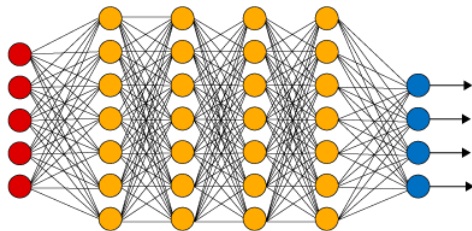
Deep neural network

Simple Neural Network



● Input Layer

Deep Learning Neural Network



● Hidden Layer

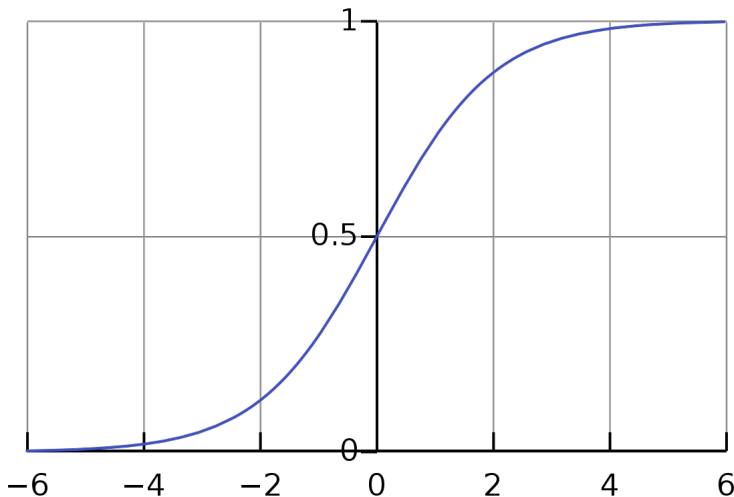
● Output Layer

Activation functions

1. Sigmoid: $\sigma(x) = \frac{1}{1 + e^{-x}}$

Activation functions

1. Sigmoid: $\sigma(x) = \frac{1}{1 + e^{-x}}$

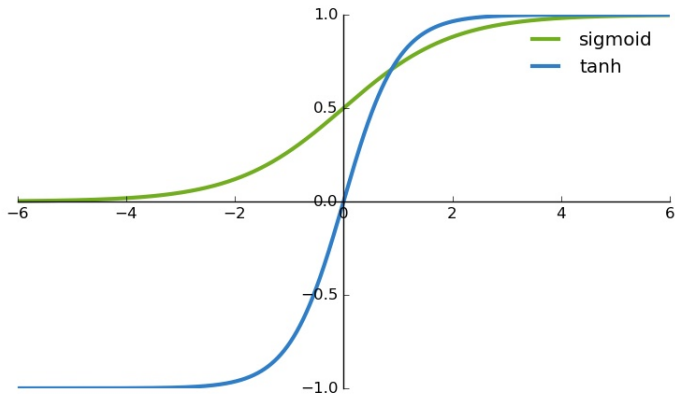


Activation functions

2. Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Activation functions

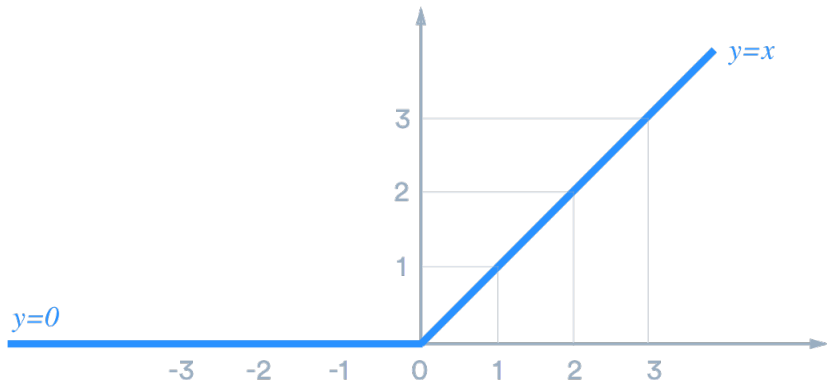
2. Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



3. Rectified linear unit: $ReLU(x) = \max(0, x)$

Activation functions

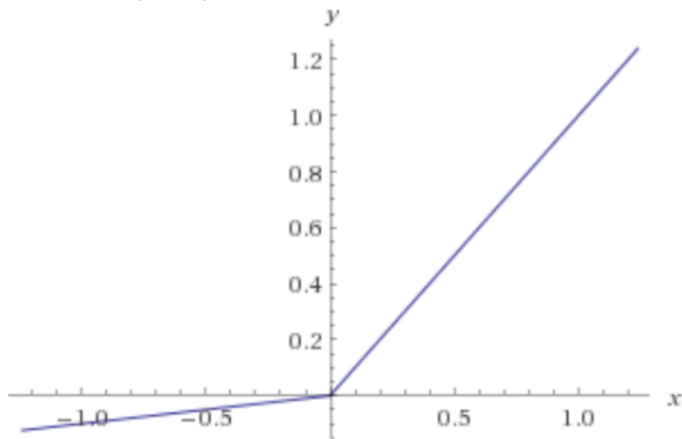
3. Rectified linear unit: $ReLU(x) = \max(0, x)$



4. Leaky ReLU: $LR(x) = \begin{cases} 0.01x, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$

Activation functions

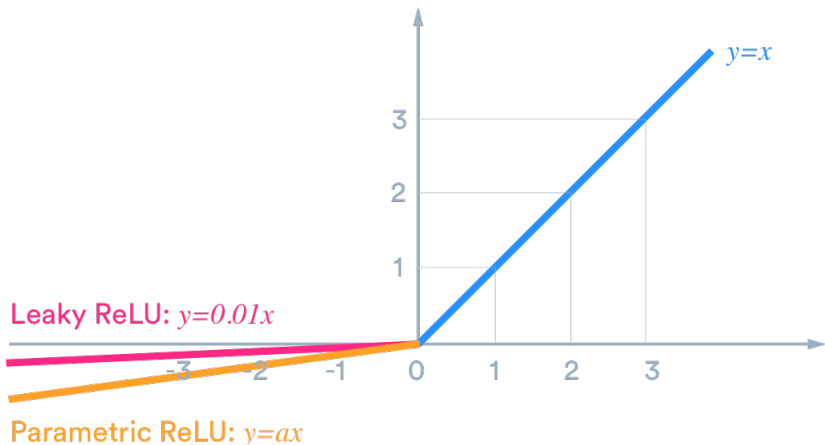
4. Leaky ReLU: $LR(x) = \begin{cases} 0.01x, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$



5. Parametric ReLU: $PR(x) = \begin{cases} ax, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$

Activation functions

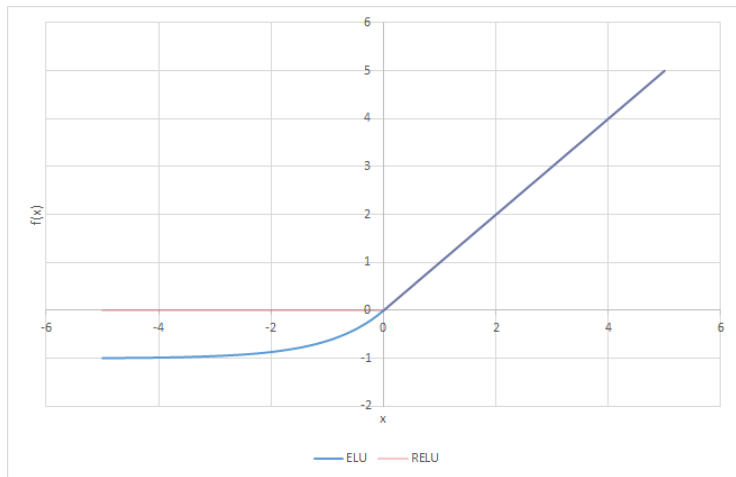
5. Parametric ReLU: $PR(x) = \begin{cases} ax, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$



6. Exponential linear unit: $ELU(x) = \begin{cases} a(e^x - 1), & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$

Activation functions

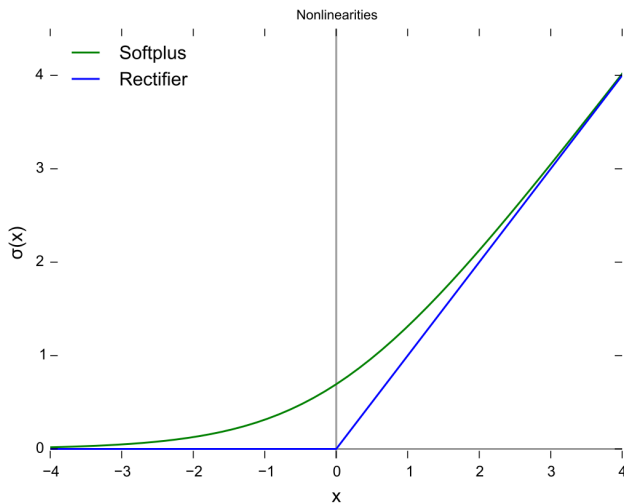
6. Exponential linear unit: $ELU(x) = \begin{cases} a(e^x - 1), & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$



7. SoftPlus: $SP(x) = \log(1 + e^x)$

Activation functions

7. SoftPlus: $SP(x) = \log(1 + e^x)$



8. Softmax: $S(x_1, x_2, \dots, x_n) = \left(\frac{e^{x_1}}{\sum_{i=1}^n e^{x_i}}, \frac{e^{x_2}}{\sum_{i=1}^n e^{x_i}}, \dots, \frac{e^{x_n}}{\sum_{i=1}^n e^{x_i}} \right)$

- 1 Why do we need activation functions?

- ① Why do we need activation functions?
- ② How should we define activation functions, for a layer or for a neuron?

Outline

- 1 What is a neural network?
- 2 Gradient Descent**
- 3 Linear and Logistic Regressions

Gradient Descent

Let $f : \mathbb{R}^k \rightarrow \mathbb{R}$ be a convex function and we want to find its global minimum.

Let $f : \mathbb{R}^k \rightarrow \mathbb{R}$ be a convex function and we want to find its global minimum. This optimization algorithm is based on the fact that the fastest decreasing direction of the function is the opposite direction of gradient:

$$x_{n+1} = x_n - \alpha \nabla f(x_n)$$

and $x_0 \in \mathbb{R}^k$ is a arbitrary point.

- 1 What is a neural network?
- 2 Gradient Descent
- 3 Linear and Logistic Regressions

Linear Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}$ be our training data.

Linear Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}$ be our training data. Consider the function

$$f(x) = f(x^1, x^2, \dots, x^k) = w^1 x^1 + w^2 x^2 + \dots + w^k x^k + b = w^T x + b.$$

Linear Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}$ be our training data. Consider the function

$$f(x) = f(x^1, x^2, \dots, x^k) = w^1 x^1 + w^2 x^2 + \dots + w^k x^k + b = w^T x + b.$$

Our aim is to find parameters $b, w^1, w^2, \dots, w^k$ such that

$$f(x_i) \approx y_i, i = 1, \dots, n.$$

Linear Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}$ be our training data. Consider the function

$$f(x) = f(x^1, x^2, \dots, x^k) = w^1 x^1 + w^2 x^2 + \dots + w^k x^k + b = w^T x + b.$$

Our aim is to find parameters $b, w^1, w^2, \dots, w^k$ such that

$$f(x_i) \approx y_i, i = 1, \dots, n.$$

We choose L^2 distance as our loss function:

$$\frac{1}{n} \sum_{l=1}^n (f(x_l) - y_l)^2.$$

- 1 Should we minimize the loss function using gradient descent?

- 1 Should we minimize the loss function using gradient descent?
- 2 Can you represent this model as a neural network?

Logistic Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \{0, 1\}$ be our training data.

Logistic Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \{0, 1\}$ be our training data. Consider the function

$$\begin{aligned} f(x) &= f(x^1, x^2, \dots, x^k) = \sigma(w^1 x^1 + w^2 x^2 + \dots + w^k x^k + b) \\ &= \sigma(w^T x + b). \end{aligned}$$

Logistic Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \{0, 1\}$ be our training data. Consider the function

$$\begin{aligned} f(x) &= f(x^1, x^2, \dots, x^k) = \sigma(w^1 x^1 + w^2 x^2 + \dots + w^k x^k + b) \\ &= \sigma(w^T x + b). \end{aligned}$$

Our aim is to find parameters $b, w^1, w^2, \dots, w^k$ such that

$$f(x_i) \approx y_i, i = 1, \dots, n.$$

Logistic Regression

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \{0, 1\}$ be our training data. Consider the function

$$\begin{aligned} f(x) &= f(x^1, x^2, \dots, x^k) = \sigma(w^1 x^1 + w^2 x^2 + \dots + w^k x^k + b) \\ &= \sigma(w^T x + b). \end{aligned}$$

Our aim is to find parameters $b, w^1, w^2, \dots, w^k$ such that

$$f(x_i) \approx y_i, i = 1, \dots, n.$$

We choose cross entropy distance as our loss function:

$$\frac{1}{n} \sum_{l=1}^n (-y_l \log f(x_l) - (1 - y_l) \log (1 - f(x_l))).$$